

5.4 - Anatomia de um pipeline maduro: stages, gates e artefatos

Cada stage com propósito, cada gate calibrado, cada artefato auditável

Stages, a coreografia de um pipeline maduro

Cada etapa com responsabilidade única e clara

- **Lint e análise estática rodam primeiro, segundos de execução, feedback imediato**
 - Falhas aqui evitam consumo de tempo nas etapas mais caras adiante
- **Testes unitários e de integração validam comportamento da aplicação**
 - Unitário cobre lógica isolada, integração cobre comunicação entre componentes
- **Análise de segurança vem antes do build, SAST e SCA são gates**
 - SAST analisa o código fonte, SCA analisa dependências externas
- **Build de imagem com tag semântica gera artefato versionado e imutável**
 - Imagem nunca é rebuilt, é promovida entre ambientes
- **Deploy em staging, smoke tests, promoção controlada para produção**

Quality gates, critérios objetivos de promoção

O que separa um pipeline rigoroso de um decorativo

- **Quality gate é regra binária, passou ou não passou, sem zona cinza**
 - Cobertura mínima de testes, número máximo de vulnerabilidades, performance de baseline
- **Gates precisam ser calibrados, restritivos demais geram bypass cultural**
 - Cobertura de 95% é desejável, exigir 95% como gate gera mocks falsos
- **Gate sem ação consequente é apenas relatório bonito**
 - Se a falha não barra a promoção, o gate não existe na prática
- **Calibrar gates é trabalho contínuo, não decisão única**
 - Métricas históricas do próprio pipeline guiam o ajuste dos limites

A evolução do pipeline da LunaLog em quatro anos

- **Pipeline inicial era um script shell de quarenta linhas em uma máquina compartilhada**
 - Pipeline atual tem nove stages, sete quality gates e artefatos auditáveis

LUNALOG

Em 2021, o pipeline da LunaLog era um arquivo `deploy.sh` com quarenta linhas, executado por quem tivesse acesso SSH naquele momento. Em 2022, Roberto extraiu os passos para um workflow do GitHub Actions, com lint e testes obrigatórios. Em 2023, Ana adicionou SAST, SCA e build de imagem com tag semântica. Em 2024, vieram smoke tests automatizados em staging e promoção controlada com aprovação técnica. Cada etapa dessa evolução foi consequência de um incidente concreto. Nenhuma foi adotada por moda, todas por dor anterior.

SAST, SCA e análise de segurança no pipeline

Segurança como gate, não como auditoria posterior

- **SAST, Static Application Security Testing, analisa o código fonte por padrões inseguros**
 - Detecta SQL injection, XSS, hardcoded secrets, criptografia frágil
 - *Ferramentas comuns, SonarQube, Semgrep, Checkmarx*
- **SCA, Software Composition Analysis, analisa dependências de terceiros**
 - Detecta CVEs conhecidas em libs, licenças incompatíveis, pacotes abandonados
 - *Ferramentas comuns, Dependabot, Snyk, Trivy*
- **DAST roda contra ambiente em execução, complementar e geralmente fora do pipeline**
- **Política de severidade define o que barra o pipeline**
 - Bloquear pipeline em CVE de severidade baixa gera ruído, não segurança

Pipeline imaturo contra pipeline maduro

Pipeline imaturo

- Stages misturados, ordem arbitrária
- Testes opcionais, podem ser puláveis com flag
- Sem análise de segurança ou rodando após o deploy
- Imagens com tag latest, sem versionamento
- Deploy direto, sem ambiente de staging fiel
- Falhas resolvidas com retry, não com diagnóstico

Pipeline maduro

- Stages ordenadas do mais barato ao mais caro
- Testes obrigatórios, gate bloqueante de promoção
- SAST e SCA antes do build, severidade calibrada
- Imagens com tag semântica, imutáveis entre ambientes
- Staging fiel à produção, smoke tests automatizados
- Falhas tratadas como sinal, retry é exceção

Gate calibrado, não burocrático

Gate que falha demais sem motivo claro vira cultura de bypass. Engenheiros começam a tratar falhas do pipeline como ruído, pulam gates com flags emergenciais e perdem o respeito pela automação. Um gate só funciona se a maioria das falhas representa um problema real que ninguém quer ignorar.

- **Reveja gates trimestralmente, ajuste limites com base em dados**
 - Métrica de saúde, taxa de falsos positivos abaixo de 10%



Pipeline com stages bem definidas e quality gates calibrados, mas quem garante que o próprio pipeline não vira ponto cego de risco? Um YAML editado direto na interface web, sem revisão, sem versionamento, é exatamente o tipo de superfície de ataque que ninguém percebe até o dia em que alguém otimiza removendo silenciosamente os testes de integração.

Pipeline como código, infraestrutura, reprodutibilidade e manutenção